

ELEE 4230/6230: Sensors and Transducers

Course Project – Semester 2026

INDOOR AIR SENSING SYSTEM

Avery Dedjoe

Table of Contents

ELEE 4230/6230E: Course Project 1

0. A Tour of 'INDOOR AIR SENSING SYSTEM' 2
1. Problem Definition and Environment and Project Constraints 3
2. Sensor Selection and Design Decisions 5
3. RAW Sensor Dynamics and Signal Processing 7
4. Capacitive Sensor (User Input System) 12
5. Other Sensors Used 14
6. Digital Logic + Algorithms 16
7. Final Design and Experimental Results 18
8. Failure Analysis 19
9. Physical Implementation 20
10. Course Reflection 21
11. References 22

Appendix 23

0. A Tour of my 'INDOOR AIR SENSING SYSTEM'

This is a portable, battery-powered monitor for the non-particulate qualities of indoor air. It reads five measurands - temperature (from two independent sensors), relative humidity, barometric pressure, and the air's gas chemistry (equivalent CO_2 and total volatile organic compounds) - and fuses four of them, live on the ESP32, into three concern-specific indices: an Asthma Index, a Sleep Index, and a Freshness Index. Each is a 0–100 score with a plain-language rating (Good / Average / Poor).

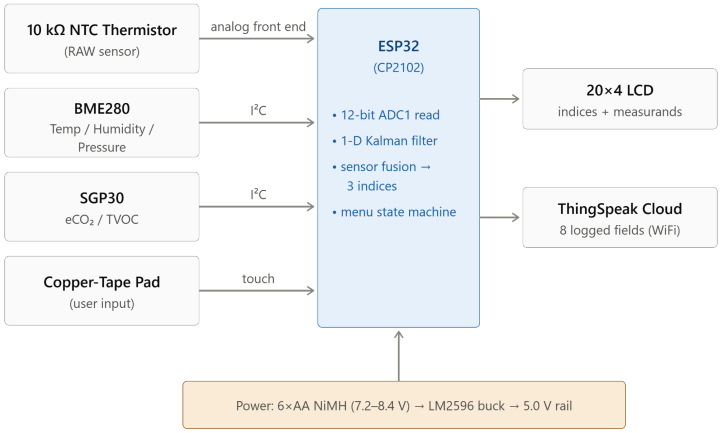


Figure 1. System block diagram: four inputs feed the ESP32, which drives an LCD and a cloud channel; a 6×AA pack regulated to 5 V powers everything.

Inputs → processing → outputs. The ESP32 linearizes and Kalman-filters the raw thermistor, reads the two I²C sensors, normalizes each measurand to a 0–100 subscore, and computes the three weighted indices every second. The user moves through the on-device menu with a homemade copper-tape touch pad (tap to cycle, hold to select); the selected index or raw measurand shows on a 20×4 LCD, and eight fields - each measurand plus the Asthma and Sleep indices - are logged to a ThingSpeak cloud channel over WiFi. The Freshness index is computed and shown on-device.

Evidence Block



Figure 2. The finished unit on a shelf, temperature being read on the LCD.

The assembled kit runs unattended on the shelf and shows a live measurand (Temperature) on the LCD. The whole system, sensing to three fused outputs, fits in one hand-sized container.

1. Problem Definition and Environment and Project Constraints

The problem

I have asthma, and the air in the room I spend the most time in has an outsized effect on how I feel and how well I sleep. When I moved into my apartment I bought a good particulate air purifier and my comfort improved right away, but a particulate purifier only filters what it can catch in a filter. It is blind to the non-particulate dimensions of indoor air: temperature, humidity, and the gas-phase chemistry of the room (volatile organic compounds, and CO_2 as a ventilation indicator). Those dimensions still change how a room feels, and nothing in my apartment measures them.

The engineering problem is therefore: measure the non-particulate properties of the air in an occupied room and turn them into information a person can act on. I originally framed this as a single "overall air-quality" number. Building it taught me that a single number is the wrong target, because different concerns weight the same measurements differently - a room cooled to 19 °C is close to ideal for sleep but can read as a cold, drying trigger for asthma. The real problem is **concern-specific fusion**, not one universal score. (My solution - the sensors and the algorithm - is developed in the sections that follow.)

Evidence Block

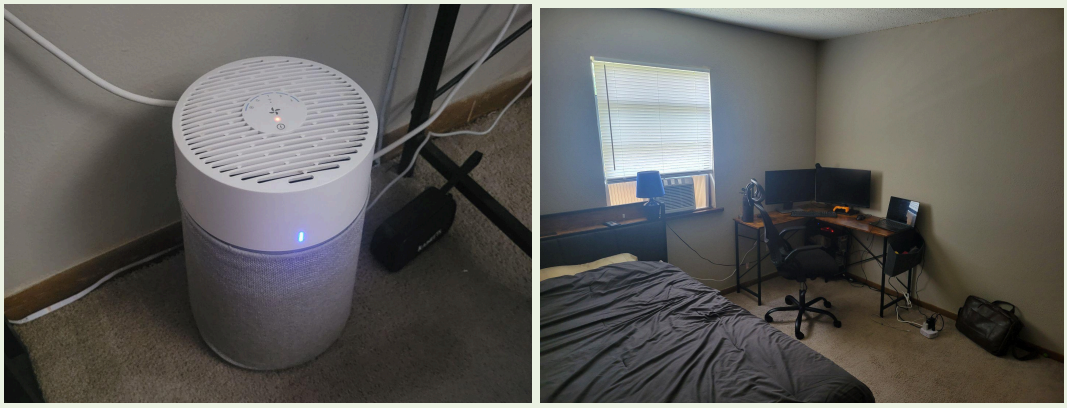


Figure 3. Left: My air purifier. Right: The Air-Sensing System's intended environment (my room)

The purifier ran continuously and handled dust, but I wondered if the air quality could be improved further. The fact that particulates are already handled is why this project measures temperature, humidity, and air chemistry instead of particulates.

Environment being measured

The system is designed for one occupied indoor room - my apartment bedroom - in Athens, GA, roughly 750 ft (≈230 m) above sea level (this elevation matters for pressure calibration, §3). The deployment environment is ordinary residential: 15–35 °C, 20–70 %RH, ~980–1030 hPa, normal household acoustic and lighting levels, and static (breadboard) mounting. The only notable electromagnetic source is the ESP32's own 2.4 GHz WiFi radio.

Safety-checklist summary. I completed the Sensors Safety Checklist for all three sensors (RAW thermistor, BME280, SGP30), one worksheet each; the full checklist is submitted separately as the Excel file. Every environmental check passed - each sensor's rated range comfortably contains the indoor range of conditions. The findings that shaped the build: a single common ground (battery negative) means no ground loops; the SGP30's metal-oxide element is poisoned by siloxanes and solvents, so it is kept away from sealants and solvent storage; the high-impedance thermistor node is kept short and Kalman-filtered against coupled noise; and the SGP30 hot-plate (~48 mA) is the dominant continuous load that sets the battery budget.

Constraints that drove design decisions

Table 1. Project constraints and how each shaped the design.

Constraint	Requirement / value	How it shaped the design
Raw, nonlinear sensor	Bare 10 kΩ NTC thermistor	Full analog front end + hardware linearization + Kalman (§3)
Two pre-packaged sensors (I²C)	BME280, SGP30	Shared 3.3 V I²C bus; no analog processing
Homemade capacitive sensor	Copper-tape pad	Tap/hold user-input control (§4)
Battery-powered / portable	6×AA NiMH → buck → 5.0 V Potential hours-long run-time	Sized on energy (Wh), not mAh (§3)
Cloud logging (IoT)	ESP32 WiFi → ThingSpeak	Forces thermistor onto ADC1 (ADC2 off during WiFi)
Kalman filter	1-D filter on raw thermistor	~20× reduction in sample-to-sample noise (§3)
"Go beyond" fusion	Blend measurands into a score	Three concern-specific indices (§6)
Temperature redundancy	Thermistor and BME280	Two physics paths agree within ~0.55 °C mean (§7)
Indoor range of interest	15–35 °C	Linearizer optimized at the 25 °C midpoint
Cost	~$88 total build	Reused on-hand passives; multipack spares (BOM)

A modeling limitation in the system

The SGP30 does not measure absolute gas concentration. Its TVOC and eCO_2 outputs are relative to a baseline it anchors against the cleanest air it has recently seen, and its eCO_2 is a derived estimate (±15 % class accuracy), not a true NDIR CO_2 measurement. My model therefore assumes I only need relative changes in air chemistry, not calibrated ppm. That is fine for "this room is worse than outside," which is what I act on - but it means the fused scores describe relative air quality against my captured outdoor nature-trail baseline, not an absolute concentration. (A second, smaller modeling limitation - the parallel-plate 1/d model for the pad - is covered in §4, where the model fails outright.)

2. Sensor Selection and Design Decisions

I picked the project first and then filtered the 100-plus sensors from the course materials against what the project actually needed. The goal of environmental air quality immediately ruled out light, motion, and distance sensors and pointed me at temperature, humidity, and a gas indicator. Within each role I compared real options against my constraints before committing.

Table 2. Air-chemistry sensor options (metrics tied to my constraints).

Option	Output	Interface	Power / speed	Verdict
MQ-135 (raw MOX)	Single analog gas level	Analog	Constant heater; slow	Raw, but strains battery; hard to calibrate
CCS811	eCO_2 + TVOC	I²C	Moderate	Workable; older/less reliable than SGP30
SGP30 (chosen)	eCO_2 and TVOC (one element)	I²C	Hot-plate ~48 mA	Two measurands, reliable, humidity-compensated

Table 3. Raw-sensor options for temperature.

Option	Signal at room temp	Linearity	Cost	Verdict
Thermocouple	µV-level	~linear	Low	Too small to amplify cleanly on a battery breadboard
RTD	Moderate	Very linear	High	Too linear - defeats the linearization exercise
NTC thermistor (chosen)	Large R swing	Strongly nonlinear	$0.80 (Lowest)	Biggest nonlinear response at room temp, cheapest

Final selection

1. **10 kΩ NTC thermistor** - the raw, nonlinear sensor. Biggest nonlinear response at room temperature, lowest cost, and a second independent physical path to temperature alongside the BME280.
2. **BME280** - pre-packaged sensor #1 (temperature, humidity, pressure). Chosen over the BMP280 because the fusion index needs humidity.
3. **SGP30** - pre-packaged sensor #2 (eCO_2/TVOC). Supplies the air-chemistry measurand that matters most for asthma.
4. **Copper-tape capacitive pad** - the homemade sensor, used as the tap/hold input to move through the on-device menu.

Evidence Block

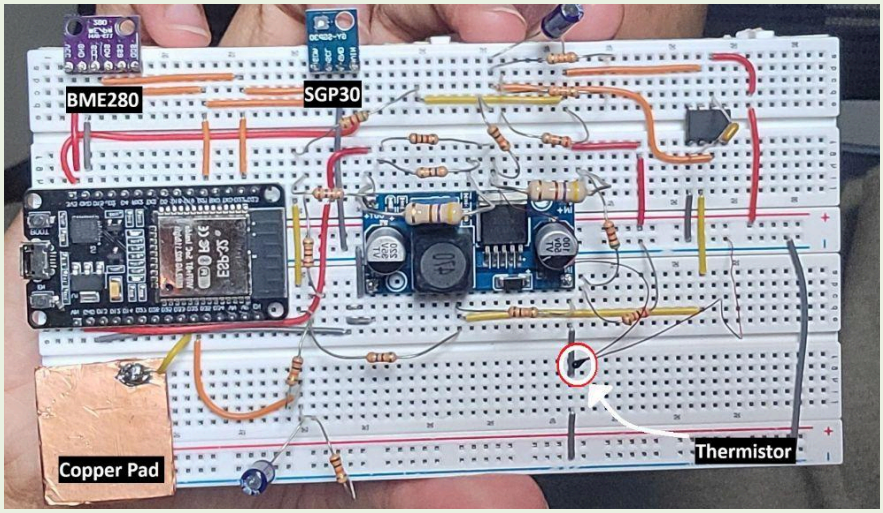


Figure 4. Bare thermistor, BME280, SGP30, and Copper Pad on breadboard - labeled.

One raw sensor and two digital sensors, chosen so the "raw nonlinear" requirement is met by the cheapest, most sensitive option (the thermistor) while the harder-to-calibrate chemistry measurement is handled by a self-calibrating digital part (the SGP30).

Design Decision Blocks

Decision 1 - SGP30 over the MQ-135 (raw MOX gas sensor). I chose the SGP30 for reliable eCO_2 and TVOC from one element over I^2C with on-chip humidity compensation. The MQ-135 also satisfies the "raw sensor" slot, but its heater draws ~150 mA continuous (vs. the SGP30's ~48 mA), outputs one uncalibrated analog gas level instead of two measurands, has no humidity compensation, and is slower than the SGP30.

Decision 2 - NTC thermistor over an RTD or thermocouple. The RTD's response is nearly linear, which defeats the raw-nonlinear-sensor-plus-linearization work this course is built around. The thermocouple puts out only microvolts, needing a high-gain, low-noise amplifier that is impractical to keep quiet on a battery breadboard. The thermistor gives the largest nonlinear swing at room temperature for the lowest cost.

Decision 3 - BME280 over the BMP280. The BMP280 drops the humidity channel. Humidity is one of the four measurands the index scores depend on (it carries meaningful weight in the Sleep and Freshness indices), so the BMP280 would have left a void that a separate humidity sensor would have had to fill.

3. RAW Sensor Dynamics and Signal Processing

3.1 Raw sensor physics and modeling

The raw sensor is a bare 10 kΩ NTC thermistor - a metal-oxide semiconducting ceramic whose conduction is thermally activated: as temperature rises, more charge carriers are freed and its resistance falls. The response is intrinsically exponential and follows the β-model:

$$R_{NTC}(T) = R_{25}\, e^{\beta\left(\frac{1}{T} - \frac{1}{T_{25}}\right)}$$

with **R_{25} = 10.21 kΩ** and **β = 3992 K** (T in kelvin, T_{25} = 298.15 K). To characterize the device I took 80 resistance-vs-temperature readings across 1.3–61 °C in several water baths, against a digital probe. A nonlinear least-squares fit (Levenberg–Marquardt) gave β = 3992 K and R_{25} = 10.21 kΩ with **R^2 = 0.9995**; inverted, the model recovers temperature to within **±0.30 °C RMS** over the full range.

Model limitation. The β-model is a two-parameter fit, so a small residual nonlinearity remains, worst at the ends of the fitted range. It is also device-specific: the printed part marking ("3590B") did not match the device I measured, which is why I use the fitted β = 3992 K rather than any nominal number. My region of interest is the 15–35 °C band an indoor room actually lives in.

Because equal temperature steps produce unequal resistance steps, the transfer function is nonlinear. I flatten it with a parallel resistor R_2 across the thermistor, which caps the cold-end resistance. The value that gives the flattest curve at the midpoint of the region of interest is:

$$R_p = R_T \frac{\beta - 2T}{\beta + 2T} = 7.56\ \text{k}\Omega \quad (T = 298.15\ \text{K})$$

I built the linearizer from resistors I had on hand (design target 7.56 kΩ), measured the actual combination at 7.46 kΩ, and used that measured value in the schematic and firmware. After the parallel resistor, a least-squares fit to the linearized data over the region of interest is:

$$R_{par} = 4.32 - 0.078\,(T - 25)\ \text{k}\Omega$$

which stays within 29 Ω of that straight line.

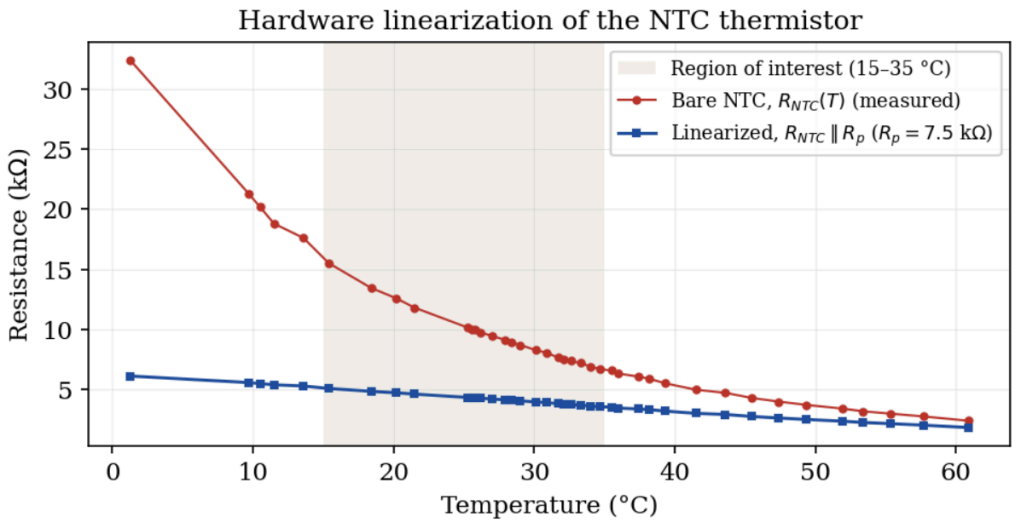


Figure 5. Measured bare NTC curve vs. hardware-linearized predicted curve. (nominal 7.5kΩ; measured 7.46 kΩ)

3.2 Signal processing path ("Signal Processing Mountain")

The forward (hardware) chain climbs temperature → resistance → voltage → 12-bit ADC count; the inverse (firmware) chain descends count → voltage → resistance → estimated temperature, inverting each stage in turn.

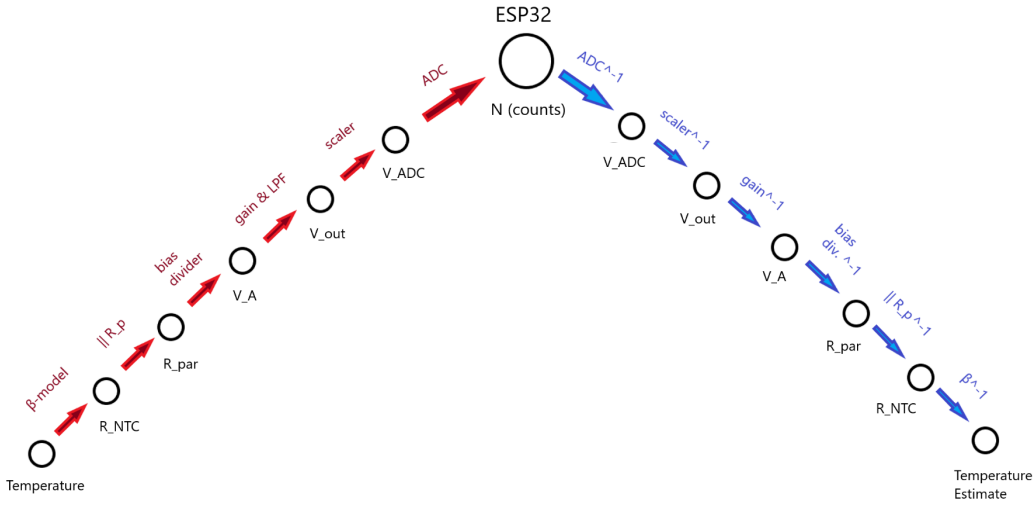


Figure 6. Signal Processing Mountain: hardware processing (red), software inversions (blue)

Forward transfer functions. The linearized resistance becomes a voltage at a bias divider, is amplified and level-shifted, scaled to protect the ADC pin, and then digitized:

$$V_A = V_{supply} \frac{R_{par}}{R_{par} + R_1}$$

$$V_{out} = V_M (1 + G) - G V_A$$

$$V_{ADC} = \frac{R_8}{R_7 + R_8} V_{out} = 0.40 V_{out}$$

$$N = \frac{V_{ADC}}{V_{fs}} (2^{12} - 1)$$

Component values: V_{supply} = 5.0 V, R_1 = 1.8 kΩ, R_2 = 7.46 kΩ, V_M = 3.50 V (from divider R_3/R_4 = 4.22 kΩ / 9.71 kΩ), G = R_6/R_5 = 84.2 kΩ / 9.7 kΩ ≈ 8.7, R_7 = 15 kΩ, R_8 = 10 kΩ, V_{fs} = 3.3 V.

1. **Parallel linearizer** turns the exponential curve into a near-straight line over the region of interest, so the inversion is well-conditioned.
2. **Bias divider** converts the linearized resistance into a voltage V_A, which swings only ~0.4 V, centered near 3.5 V.
3. **Op-amp gain + level shift (TL071CP)** references that swing to V_M ≈ 3.50 V and multiplies it by G ≈ 8.7, stretching the ~0.4 V sensor swing across a usable portion of the ADC range for more real resolution. The same stage is an active first-order low-pass filter (C_2 // R_6) that passes the slow temperature signal and rejects ambient noise.
4. **Output scaler** (×0.40) guarantees $V_{ADC} \leq 3.3$ V so the ESP32 input pin is never over-driven.
5. **ADC** - read on GPIO32, which is on ADC1: a required choice, because the ESP32 disables ADC2 whenever WiFi is active.

3.3 Final implementation (circuit and power)

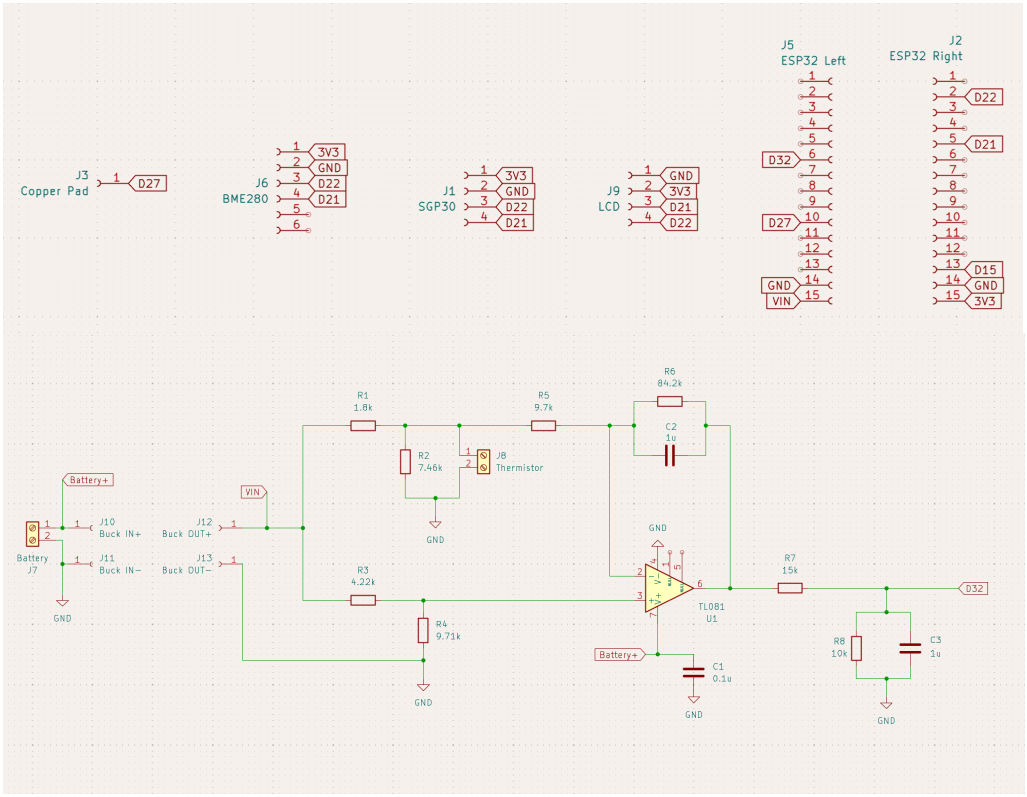


Figure 7. Upper: pre-packaged modules on female headers; lower: the thermistor analog chain. (The schematic symbol U1 carries the library's TL081 label; the socketed part is a TL071CP.)

Circuit. The analog front end is built around a single TL071CP in a DIP-8 socket. The bias divider, R_1 = 1.8 kΩ in series with the thermistor paralleled by the linearizer R_2 = 7.46 kΩ, converts the linearized resistance into the voltage V_A. V_A drives the op-amp's inverting input through the input resistor R_5 = 9.7 kΩ, with R_6 = 84.2 kΩ as feedback, giving a gain of $R_6/R_5 \approx 8.7$; C_2 = 1 µF across R_6 makes that same stage an active low-pass filter. The non-inverting input is held at the reference V_M = 3.50 V by a second divider, R_3 = 4.22 kΩ and R_4 = 9.71 kΩ, off the 5 V rail: the stage amplifies the sensor swing about V_M rather than about ground and the output follows $V_{out} = V_M(1 + G) − G \cdot V_A$. That output passes through the divider R_7 = 15 kΩ / R_8 = 10 kΩ, which scales it by ×0.40, with C_3 = 1 µF as a final low-pass, so V_{ADC} never exceeds 3.3 V, and is read on GPIO32 (ADC1). C_1 = 0.1 µF decouples the supply. This accounts for all eight resistors R_1–R_8 and all three capacitors; the pre-packaged modules sit on 2.54 mm female headers.

Power (battery → regulation → sensors). The system runs on six AA NiMH cells (7.2 V nominal, 8.4 V fresh). An LM2596 buck converter produces a stable 5.0 V that powers the ESP32 and excites the sensor's bias divider and reference - so the ADC reading is proportional to that same rail and does not drift as the pack discharges. The op-amp is supplied directly from the raw battery rail (7.2–8.4 V) for output headroom in a single-supply configuration. I chose 6×AA over a 9 V battery on energy, not capacity: 7.2 V × 2800 mAh ≈ 20.2 Wh versus a 9 V's ~4.5 Wh - about 4–5× the runtime - and the pack's lower internal resistance (~0.2–0.3 Ω vs ~1–2 Ω) keeps the rail stiff under WiFi current spikes. The LM2596's ~30 mV ripple is attenuated by the divider ratios and the Kalman filter.

Calibration. The β-model fit is the thermistor's bench calibration; residual offset is removed in firmware by comparing the thermistor to the BME280 at a known temperature. The BME280 itself was calibrated in CP2: temperature against an ice-water bath (−1.201 °C offset), pressure against an elevation-adjusted forecast (Athens ≈ 750 ft → subtract ~27 hPa; −1.3 hPa offset), and humidity against a saturated-salt reference (75 %RH; +4.73 % offset).

3.4 Filtering and Kalman implementation

I apply a 1-D Kalman filter to the recovered temperature, modeling temperature as a random-walk state. The tuned parameters are **process noise Q = 0.0008** and **measurement noise R = 0.15**, sampling at 5 Hz; I set them by trial-and-error to knock down ADC noise without smearing real temperature change.

Evidence Block

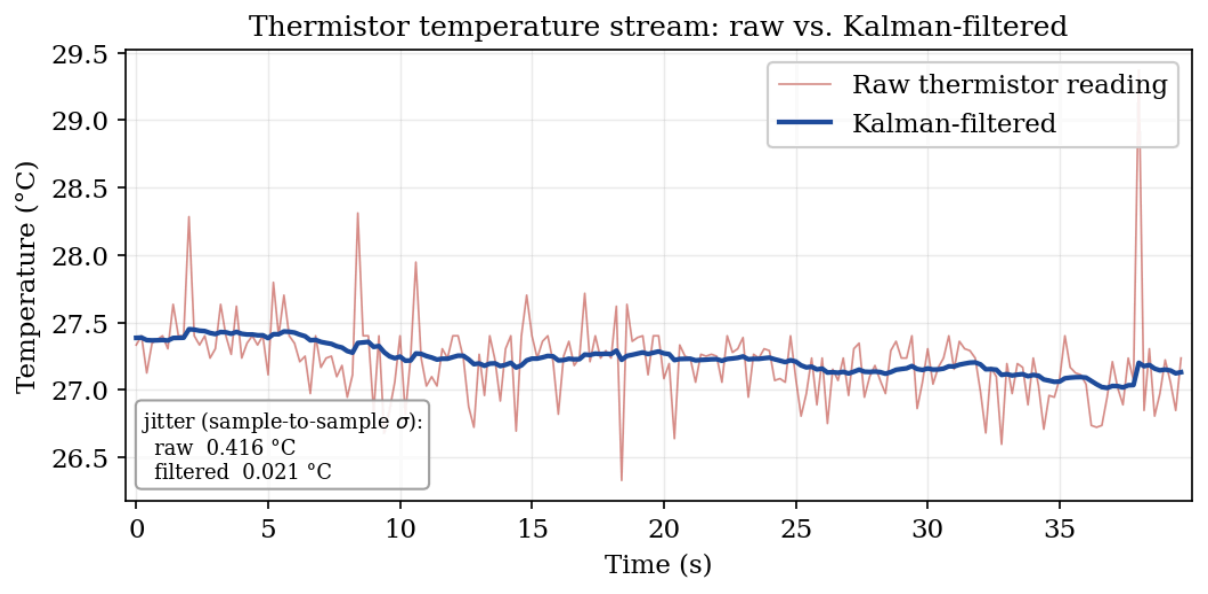


Figure 8. ~40 s of thermistor data at steady room temperature, before (noisy) and after (smooth) the Kalman filter.

Over the capture, sample-to-sample jitter dropped from ±0.42 °C raw to ±0.02 °C filtered - about a 20× reduction - while the filter still tracked a real ~0.4 °C drift instead of flattening it.

Table 4. Thermistor error budget over the 15–35 °C region of interest.

Error source	Magnitude
Sensor β-model fit	±0.30 °C RMS (0.65 °C max)
Hardware linearizer residual	±0.87 °C max
ADC noise (raw)	±0.42 °C sample-to-sample
ADC noise (after Kalman)	±0.02 °C
Battery-voltage drift	~0 (proportional to 5V regulated rail)
Net, after calibration & Kalman	~±0.9 °C RMS

4. Capacitive Sensor (User Input System)

4.1 Physics and construction

The homemade sensor is a 2.5 × 2.5 cm square of copper tape on a cardboard base, with a wire soldered at one corner running to GPIO27 on the ESP32. An approaching finger and the pad act as the two plates of a parallel-plate capacitor:

$$C = \frac{\varepsilon_0 \varepsilon_r A}{d}$$

so the pad's capacitance grows as the finger-to-pad separation d shrinks. The ESP32's on-chip touch peripheral makes the pad the timing capacitor of an internal relaxation oscillator and counts its oscillations; more capacitance lengthens each charge cycle and lowers the count. The peripheral reports that count as an integer N that sits at a baseline when untouched and falls as a finger approaches:

$$N(d) = N_0 - k\,C(d) = N_0 - \frac{K}{d}$$

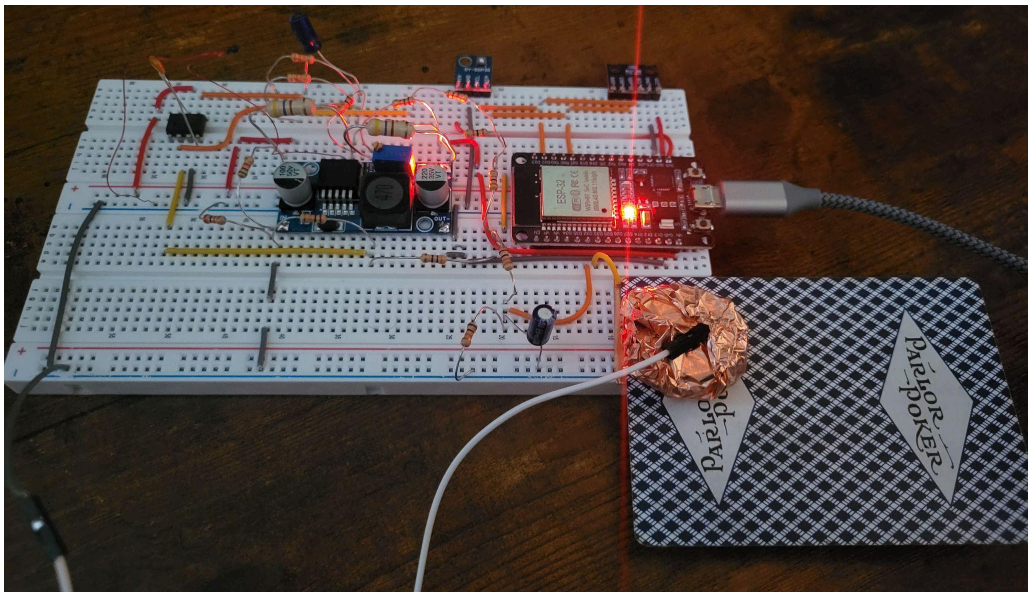


Figure 9. The copper pad & data-collection rig

Characterization. I controlled distance by stacking playing cards (54 cards = 17 mm, so 0.315 mm/card) under a coin stack wrapped in copper foil; the foil made the weight a grounded conductor, mimicking a fingertip without the pressure and contact-area variability of a real finger. I averaged three reads at each of 14 distances. The untouched baseline was $N_0 = 76$; a control of all 54 cards with no weight averaged 70.7, so the cards themselves add only ~5 counts. A nonlinear least-squares fit over 0.3–17 mm gave:

$$N(d) \approx 63 - \frac{7.5}{d}$$

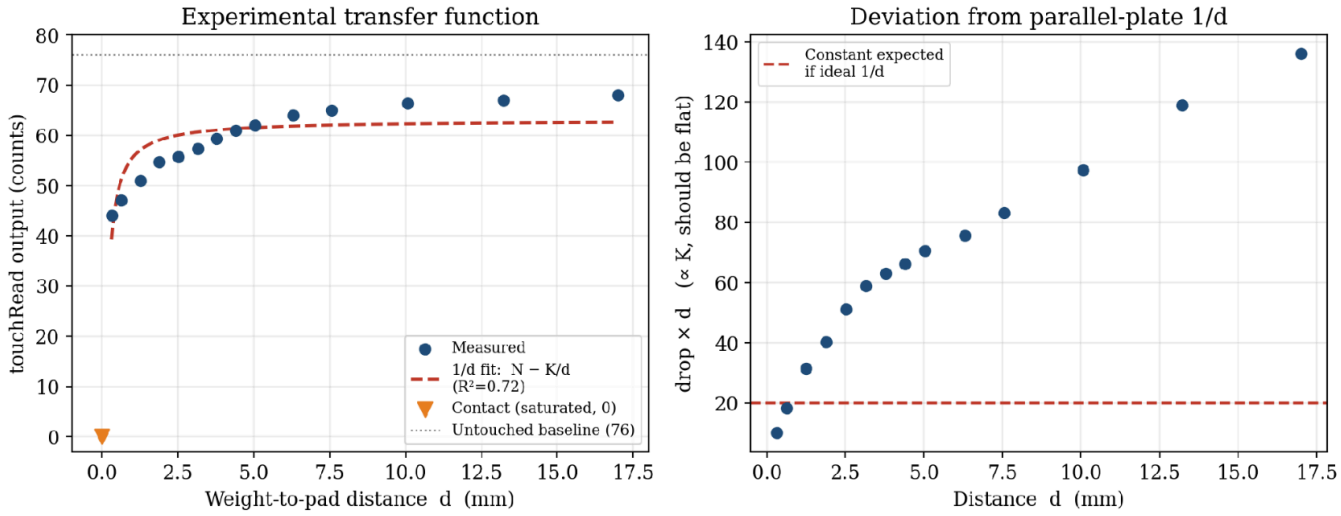


Figure 10. Left: Measured transfer function vs the ideal (1/d) fit . Right: curve showing drop × d ($drop = N_0 - N(d)$); in parallel-plate theory this would be constant.

Theory vs. reality (modeling limitation). The 1/d model captures the trend but diverges badly at the smallest and largest separations. The parallel-plate formula only counts the field directly between the plates; in reality the field fringes around the pad's edges and the pad couples to nearby conductors, so the peripheral sees capacitance even when the weight is far away. The pad is therefore a poor absolute distance sensor but an excellent touch/no-touch detector - which is why I use it as a tap/hold control rather than a proportional slider.

4.2 Signal processing and implementation

The pad is read by touchRead(27). The firmware conditions that raw count in three stages before it is classified as a gesture:

1. **Averaging.** Each raw count jitters by a few LSB, so every sample is the mean of 16 consecutive reads, cutting the noise to about ±1 count.
2. **Adaptive baseline + relative threshold.** Rather than the fixed threshold used in CP4, the final firmware learns the resting baseline at boot (32 averaged reads with no finger on the pad) and then lets it track slow drift - whenever the pad reads clearly untouched, the baseline creeps toward the resting level (baseline += (read − baseline)/8). A touch is registered when the averaged count falls at least 12 counts below the current baseline. This rejects the slow drift in the pad's resting capacitance (lid flexing, humidity, trace capacitance) that a fixed threshold cannot.
3. **Gesture detection.** A small state machine times each press. A release before 1.5 s is a tap; crossing 1.5 s while still held fires a hold. Taps cycle through menu items; holds select or go back.

On the PCB the pad trace is kept short because trace capacitance adds directly to the pad baseline; on the breadboard the pad lead does run near the 5 V and ground leads (although this has no effect on performance, possibly due to the adaptive baseline).

4.3 System integration

The pad is the system's only user control. It drives a three-state menu on the LCD: a top menu (tap cycles Asthma → Sleep → Freshness → Measurands, hold selects), an index view (tap cycles between the three indices, hold returns), and a measurand view (tap cycles the six raw readings - both temperatures, humidity, pressure, eCO_2, TVOC - hold returns). The same pad both selects which fused index the user is watching and exposes the underlying measurements behind it; the values it displays are the filtered thermistor, the BME280 channels, the SGP30 channels, and the computed indices.

4.4 Limitations and performance

Because of fringing and coupling, the pad cannot report how close a finger is, only whether one is in contact - the design consciously trades proportional sensing for a reliable binary input. With the 16-sample average giving ~±1 count of noise and a 12-count trip margin, the pad has not produced a false trigger in normal use, and the adaptive baseline keeps it reliable as the enclosure and room humidity change through the day.

Evidence Block

The measured pad characterization in Figure 10 shows the untouched count sitting at a baseline of 76 and dropping steeply as a grounded object approaches - at the closest spacing it falls to roughly 45, far past the 12-count trip margin in the firmware. That margin is about twelve times the ±1 count of noise left after 16-sample averaging, which is why a real touch registers cleanly and the pad has not produced a false trigger in normal use.

5. Other Sensors Used

5.1 BME280 (temperature / humidity / pressure)

The BME280 is a single Bosch MEMS package holding three transducers - a bandgap/resistive temperature element, a capacitive polymer humidity element, and a piezoresistive MEMS pressure membrane - read over I²C at address 0x76. In this project it is the pre-packaged temperature reference (redundant with the thermistor) and the sole humidity and pressure source.

Output characteristics: temperature ±0.5 °C at 25 °C (±1.0 °C over 0–65 °C), 0.01 °C resolution; humidity ±3 %RH, 0.008 %RH resolution; pressure ±1 hPa absolute (±0.12 hPa relative), 0.18 Pa resolution; 20-bit internal conversions.

Internal signal processing (inferred from datasheet): the raw 20-bit ADC codes are corrected on-chip against factory trim coefficients stored in non-volatile memory, and the driver applies Bosch's compensation polynomials to return calibrated T/RH/P. What looks like a clean digital reading is really a trimmed, polynomial-linearized measurement; the sensor is doing the same kind of inversion I do by hand for the thermistor, just in silicon.

Expected vs. actual (behavior in this project): I calibrated the BME280 against references in CP2 - an ice-water bath put temperature 1.20 °C high, an elevation-adjusted forecast put pressure 1.3 hPa high, and a saturated-salt chamber put humidity 4.73 % low, all removed as fixed offsets.

To characterize noise I took 100 temperature readings under constant conditions: mean 25.978 °C, standard deviation 0.210 °C. That 0.21 °C is the BME280's temperature noise floor - an order of magnitude larger than the Kalman-filtered thermistor's ±0.02 °C sample-to-sample at 5 Hz, though over longer intervals the thermistor path shows more variation than the BME280 (Fig. 16). Regressing BME280 pressure against a weather station gave y = 0.9164x + 58.8; the slope below 1 shows the two instruments track proportionally but not identically; a fixed offset corrects the bias but not the residual scale.

Limitation: pressure is accurate in a relative sense (±0.12 hPa) but its ±1 hPa absolute spec and the manual elevation correction mean I treat it as a trend indicator, not a survey-grade barometer.

5.2 SGP30 (eCO_2 / TVOC)

The SGP30 is a Sensirion metal-oxide (MOX) gas sensor: a heated SnO_2 film whose surface resistance changes as VOCs adsorb and react. On-chip electronics turn that resistance into a TVOC reading (ppb) and a derived equivalent CO_2 (ppm) from the same element, over I²C at 0x58. It provides the air-chemistry measurand primary to the Asthma and Freshness indices.

Output characteristics: eCO_2 400–60000 ppm, TVOC 0–60000 ppb; ±15 % of reading after the baseline is established; mandatory 1 s sampling; ~15 s init, ~15 min settle, 12 h first baseline; the hot-plate draws ~48 mA, the system's largest continuous load.

Internal signal processing (inferred from datasheet): the MOX resistance is ratioed against an internal baseline resistance that the on-chip algorithm continuously re-estimates as the cleanest recent air; eCO_2 is then inferred from the VOC/H_2 signature rather than measured directly (it is not an NDIR CO_2 sensor). I feed it absolute humidity computed from the BME280 so it can compensate the known humidity dependence of MOX readings.

Expected vs. actual (behavior in this project): I logged ~100 s of quiet room air, breathed on the sensor for ~30 s, then let it recover. Undisturbed it read the 400 ppm eCO_2 / ~200 ppb TVOC floor; on the breath eCO_2 rose to ~520 ppm and TVOC roughly tripled to ~600 ppb; both decayed back over ~30 s, with TVOC dipping slightly below its start as the floating baseline re-settled. The transient is exactly the shape expected for a MOX sensor responding to a VOC source.

Evidence Block

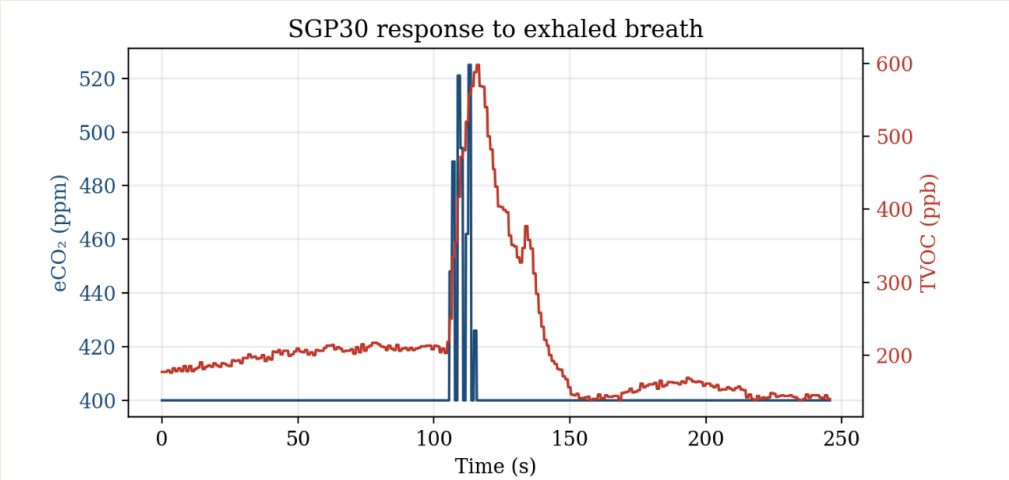


Figure 11. SGP30 response to exhalation (eCO_2 and TVOC vs time).

From the trace, a ~30 s breath moved TVOC from ~200 to ~600 ppb (roughly 3×) and eCO_2 from 400 to ~520 ppm, then both relaxed to baseline in ~30 s - confirming the sensor responds quickly and reversibly to a real VOC source rather than sitting at a fixed number.

Limitation: every SGP30 value is relative to its baseline and eCO_2 is a ±15 % derived estimate, so the sensor reports change, not calibrated concentration - the largest constraint on what the fused indices can claim.

6. Digital Logic + Algorithms

6.1 System flowchart

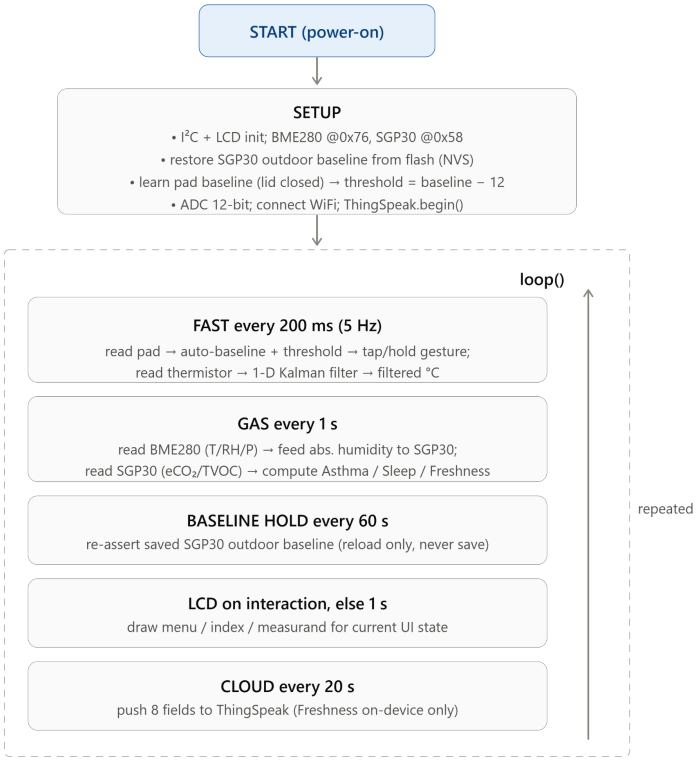


Figure 12. Firmware flow: a one-time setup followed by five independently timed tasks (task names as they appear in the firmware) in the main loop.

6.2 Algorithm descriptions

Sensor reads. The thermistor inverse chain recovers temperature from the ADC count by inverting each hardware stage of §3 in turn; the BME280 and SGP30 are read through their vendor libraries at their required cadences (the SGP30 strictly every 1 s). The pad is read and conditioned as in §4.

Kalman integration. The recovered thermistor temperature is passed through the 1-D random-walk Kalman filter (Q = 0.0008, R = 0.15) inside the 5 Hz fast loop, so every displayed and logged thermistor value is already de-noised.

"Go beyond" fusion. Every measurand is first normalized to a 0–100 subscore by a trapezoidal scoring function: full marks are awarded inside an ideal zone [x_{lo}, x_{hi}], falling off linearly to zero at a distance w past whichever edge is nearer.

$$s(x) = 100 \quad \text{for} \quad x \in [\,x_{lo},\ x_{hi}\,]$$

$$s(x) = \max\left(0,\ 100\left(1 - \frac{d}{w}\right)\right) \quad \text{otherwise}$$

The ideal zones and fall-offs are index-specific engineering choices, informed by the published guidance in §11. Because they differ per concern, the same measurement scores differently for each index - e.g., 19 °C is inside the Sleep temperature zone but below the Asthma zone.

Table 5. Implemented ideal zones [low, high] and fall-off w, per index and measurand (from firmware).

Measurand	Asthma [lo, hi] (w)	Sleep [lo, hi] (w)	Freshness [lo, hi] (w)
Temp (°C)	[20, 25] (5)	[16, 19] (4)	[18, 26] (8)
Humidity (%RH)	[40, 50] (15)	[40, 60] (15)	[30, 60] (20)
eCO_2 (ppm)	[400, 800] (800)	[400, 800] (600)	[400, 800] (1200)
TVOC (ppb)	[0, 150] (500)	[0, 150] (850)	[0, 150] (350)

Each index is then the weighted average of its four subscores, weights summing to 1. Freshness shown as a worked example:

$$I_{Fresh} = 0.45\,s_V + 0.45\,s_C + 0.05\,s_T + 0.05\,s_H$$

Table 6. Fusion weights per measurand (columns sum to 1).

Measurand	Asthma	Sleep	Freshness
TVOC	0.35	0.10	0.45
eCO_2	0.25	0.25	0.45
Temperature	0.20	0.35	0.05
Humidity	0.20	0.30	0.05

The weights encode domain logic: eCO_2 is weighted below TVOC for asthma because it is a ventilation proxy rather than a direct irritant; TVOC is weighted smallest for sleep because there is little evidence linking it to sleep quality; Freshness leans almost entirely on the two gas channels. Each 0–100 index maps to a rating: >79 Good, 50–79 Average, <50 Poor.

6.3 Design decision - µC

Chose the ESP32; the worse alternative is an Arduino Uno (ATmega328P). The Uno has no WiFi (the IoT-logging constraint would need a separate module), no capacitive touch peripheral

(the homemade pad would need an external timing circuit), and only a 10-bit ADC versus the ESP32's 12-bit: a lower resolution on the raw-sensor whose signal path is already built and calibrated by hand. The ESP32 also gave the specific ADC1/ADC2 vs. WiFi behavior I designed around in §3.

Evidence Block



Figure 13. All three indices measuring the same air: Asthma = 75/100, Freshness = 91/100, Sleep = 51/100

For one set of identical inputs the firmware reports three different scores (e.g., Sleep and Asthma diverging by tens of points), which is only possible because each index applies its own zones and weights - evidence the fusion differentiates concerns rather than relabeling one number.

7. Final Design and Experimental Results

Three results, each from the built system, each tied back to a goal set in CP1.

Result 1 - Kalman filter turns the raw thermistor usable

Result: sample-to-sample jitter fell from ±0.42 °C to ±0.02 °C (~20× reduction) while a real ~0.4 °C drift was preserved (Fig. 8, §3.4). **Interpretation:** the filter removes ADC noise, not signal, turning a twitchy raw reading into a stable one. **CP1 goal:** CP1 required a Kalman filter on the raw thermistor and a reliable temperature measurement - both satisfied.

Result 2 - The fused indices separate concerns (GO BEYOND impact)

Result: Carrying the unit from my bedroom into the kitchen after cooking dropped Asthma by 47 points and Freshness by 44, but Sleep by only 21. **Interpretation:** the same air scored three different ways because each index weights the four measurands differently. **CP1 goal:** CP1 set out to fuse the measurements into something actionable, and this returns three different actionable answers from one set of readings.

Table 7. Fused indices for the same device in two rooms.

Index (Scores/100)	Bedroom	Kitchen (post-cooking)	Change
Asthma	86	39	−47
Sleep	65	44	−21
Freshness	97	53	−44

Evidence Block

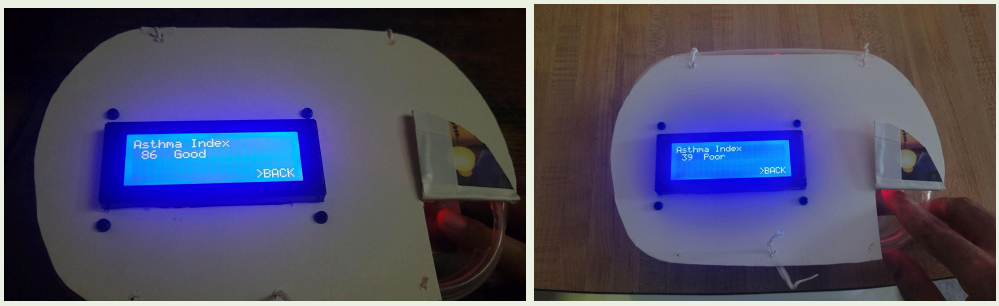


Figure 15. Left: Bedroom Asthma Index. Right: Kitchen Asthma Index (after cooking).

Readings were taken an hour apart, with all values in Table 7 having been read directly off the screen. The bedroom Sleep reading of 65 is worth noting on its own: it sits exactly at the ceiling that the scoring model predicts for any room at or above 23 °C, independently confirming the failure analyzed in §8.

Result 3 - Two-physics temperature redundancy

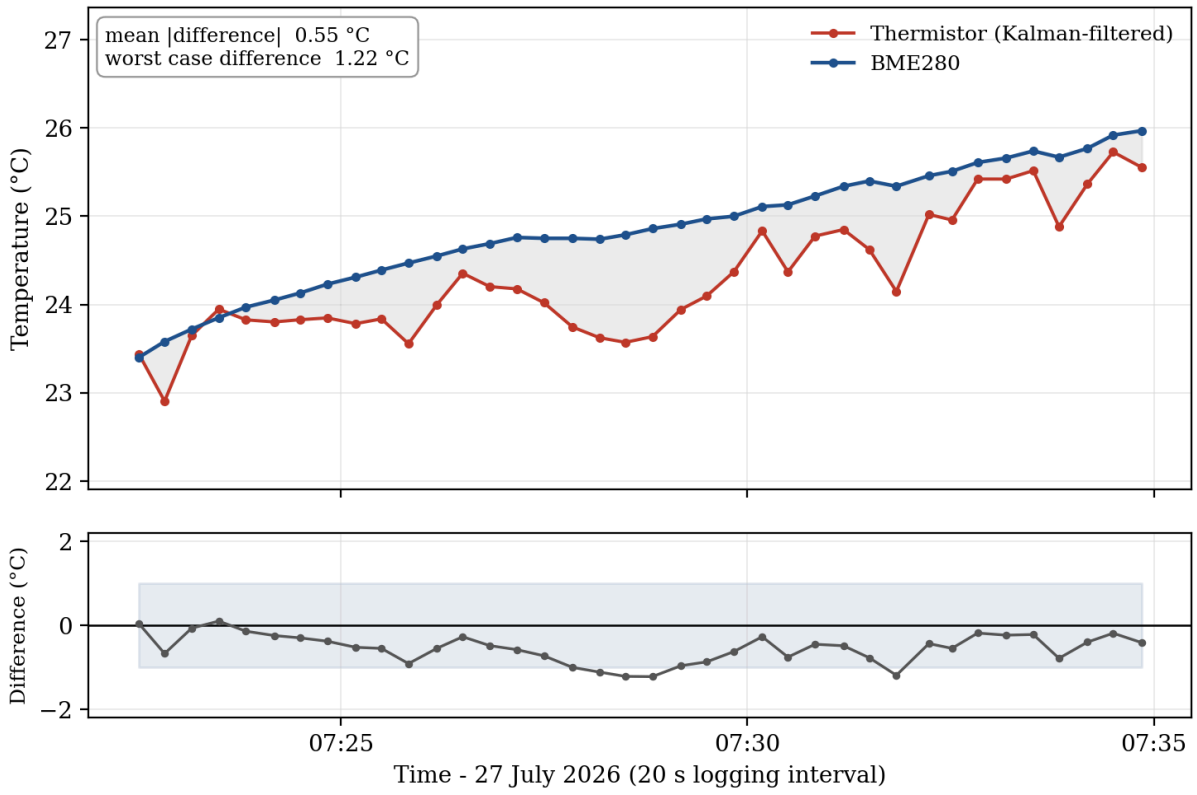


Figure 16. Thermistor (Kalman-filtered) and BME280 temperatures over 12 minutes on 27 July. The two paths agree to within a mean difference of 0.55 °C, worst case 1.22 °C; the shaded band marks ±1 °C.

Result: Over a 12-minute window on 27 July (38 samples at 20 s intervals), the Kalman-filtered thermistor and the BME280 agreed to within a mean difference of 0.55 °C, with a worst-case

separation of 1.22 °C and a spread of 0.34 °C. **Interpretation:** two sensors working on entirely different physics - the thermally activated resistance of a semiconducting ceramic against a factory-trimmed MEMS/bandgap element - independently converge on the same room temperature, and the mean difference sits inside the ~±0.9 °C RMS accuracy predicted for the thermistor chain in §3, Table 4. That is what makes the redundancy useful: a fault or drift in either path would show up immediately as the two curves separated. **CP1 goal:** CP1 required temperature redundancy across two independent physical principles; this quantifies how closely they actually track.

8. Failure Analysis

While troubleshooting the system I became concerned that the Sleep Index wasn't functioning properly because it never rose above 65. Eventually I realized that I had designed it not to.

What I expected. I expected the Sleep Index to behave like the other two indices - moving up and down across a usable range as conditions in my bedroom changed. I thought I could watch it respond to sanity-check that part of the fusion.

What actually happened. The score got stuck. The Sleep Index sat pinned near 65 (Fig. 17) no matter what else changed in the room, and I initially read that as a bug in the scoring code.

Why my assumption failed. The model was working correctly; my assumption about my own room was wrong. The Sleep Index scores temperature against an ideal zone of 16–19 °C with a 4 °C fall-off, so the temperature subscore reaches zero at any room temperature of 23 °C or above. Temperature carries a weight of 0.35 in that index, so once the room is that warm the index is mathematically capped at 65 - even with humidity, eCO_2, and TVOC all scoring perfectly. My bedroom was simply above that line essentially all the time, and the flat reading was the model telling me so. The thresholds themselves were correctly sourced; what I had not appreciated was that the temperature in my room was constantly above 23 °C, even with my AC running.

What I changed. Once I understood the index was temperature-gated rather than broken, I started running the AC in my bedroom at night at a much colder setting. The room still didn't drop far enough below the 23 °C cutoff for the Sleep Index to move into the "Good" range (22.8 °C → 66.8/100 → "Average"), but I have slept better since. The measurement changed my attitude towards setting my AC at night, but it also taught me to check a fused score's individual subscores before trusting or doubting the total.

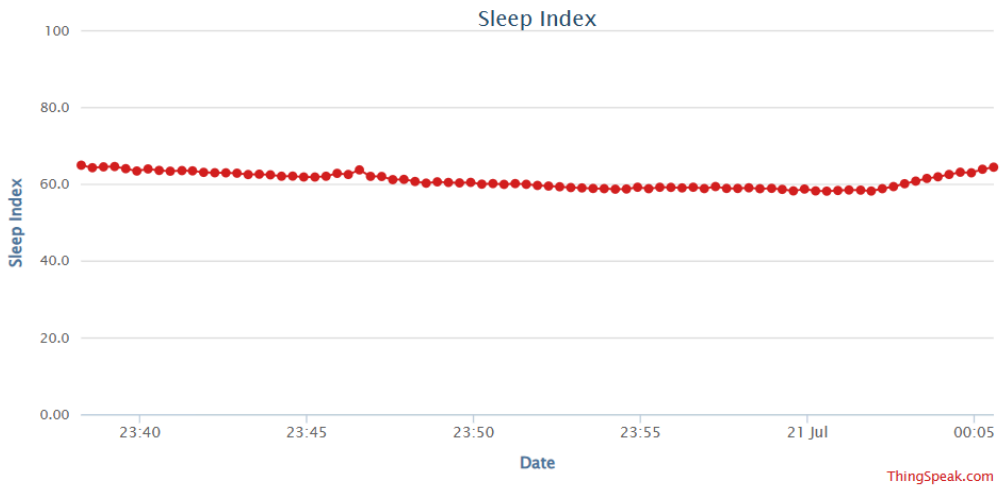


Figure 17. Sleep Index logged over 27 minutes on the night of July 20, plotted on the full 0–100 scale. The index never rises above 65 since a room at or above 23 °C drives the temperature subscore to zero and that subscore carries a weight of 0.35.

9. Physical Implementation

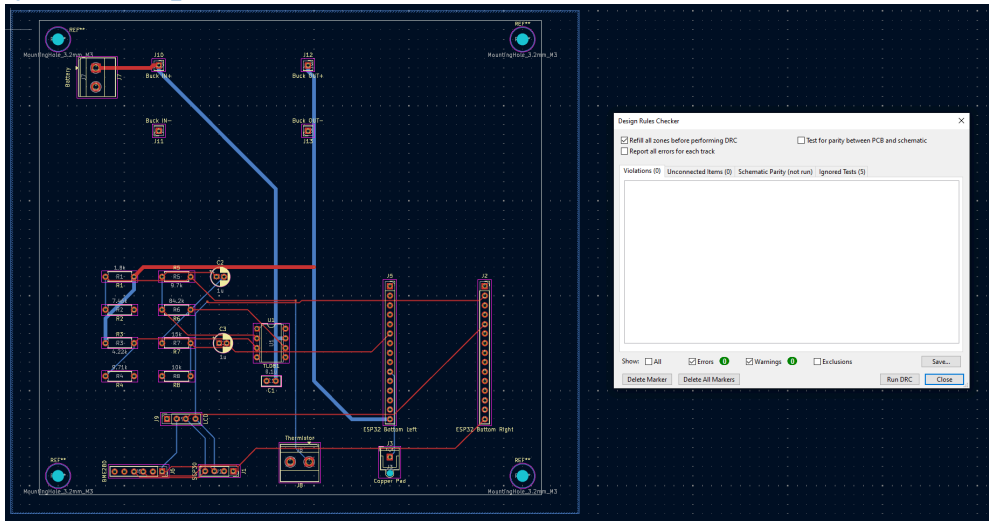


Figure 18. Rev-2 PCB designed in KiCad (clean DRC): modules on female headers, bare parts are through-hole, supply and analog sections at opposite edges.

PCB. Because the ESP32, BME280, SGP30, LM2596, and LCD backpack are modules, the board hosts them on 2.54 mm female headers; the bare parts (R_1–R_8, C_1–C_3, and the TL071CP in a DIP-8 socket) are through-hole. Every wire that leaves the board lands in a screw terminal (battery, thermistor) or a JST connector (touch pad) so nothing can vibrate loose. Routing keeps the switching supply away from the analog chain: the LM2596 and battery input sit at one board edge and the TL071CP front end at the opposite edge, a continuous ground pour on the back gives a low-impedance return and shielding, the 5 V/battery traces are widened to 1 mm for current, and the pad trace from GPIO27 is kept short because trace capacitance adds directly to

the pad baseline. The board passes DRC; the working Rev-1 system runs on the breadboard it was validated on.

Enclosure. Rev 1 is a rigid tupperware box that closes fully. Ventilation openings are cut right next to the BME280 and SGP30 so they sample moving room air instead of stale air trapped in the box; the same openings expose the USB port and the touch pad. The LCD shows through a cut in the lid, the battery pack and breadboard are pressure-fit with paper-towel padding, and twist-tie hinges let it open for service. Rev 2 is planned as a 3D-printed case with the PCB on standoffs. Power is unchanged: 6×AA → LM2596 buck → 5 V rail.

Physical limitations found in testing. My first enclosure layout put the gas sensors deep inside the sealed box, and they read stale, unchanging air - the box itself was buffering the measurement. Cutting ventilation openings directly adjacent to the BME280 and SGP30 fixed it, but it means the sensors must sit at the enclosure wall, which constrains the Rev-2 mechanical design.

Evidence Block

From the PCB image above I can highlight the design intent it enforces: eight discrete resistors and the DIP-8 op-amp socket for the analog front end, the LM2596 and battery input isolated at the upper edge, and the analog stage at the bottom edge. That physical separation is desirable for keeping buck switching noise out of any high-impedance sensor nodes.

10. Course Reflection

Project reflections

Most-incorrect assumption. That outdoor air would automatically make the cleanest reference. The SGP30 reports everything relative to a baseline, so I anchored mine outdoors on the assumption that outdoor air is the cleanest air the sensor would see. It wasn't - when I carried the unit through Driftmier, TVOC read zero everywhere, because Driftmier's air was at least as clean as the outdoor air I had anchored to. The sensor was fine and the readings were correct; my reference was the issue. It introduced me to the reality of relative measurements.

Where theory deviated from reality. The capacitive pad's parallel-plate model. Theory said $N \propto 1/d$; the real pad diverged badly at both ends because fringing fields and coupling to nearby conductors dominate - enough that I gave up on distance sensing and used the pad as a touch/no-touch input instead.

A design decision that limited performance. Choosing the SGP30 (a derived, baseline-relative MOX sensor) over a true NDIR CO_2 sensor. It was the better choice on cost, power, and getting two measurands from one part, but it permanently limits the system to relative air chemistry and a ±15 % eCO_2 estimate - the indices can say "this room is worse than outside," never "this room is X ppm CO_2."

What I would NOT do again. Building the analog front end out of series and parallel combinations of whatever resistors I had on hand. It worked - I measured each combination and carried the real values into the schematic and firmware - but it left me documenting awkward numbers like 7.46 kΩ and 84.2 kΩ instead of clean specified parts, and it makes the design

harder for anyone else to reproduce. Next time I would specify exact 1 % values up front and buy them.

Lecture reflections

The course's throughline - that a "sensor" is mostly signal processing - felt clearest while working with the thermistor: the raw device is about as simple as a resistor, but linearization, biasing, gain and level-shifting, anti-alias filtering, ADC scaling, and inversion quickly felt complicated to deal with. The Kalman lecture gave me a filter that could be tuned according to two constants, Q and R, between which noise and responsiveness are traded. The statistics lectures (mean and standard deviation, regression, the Scheffé band) gave me the tools/vocabulary to characterize noise floors and calibrate against physical references instead of guessing. And the safety-checklist lecture forced me to review each component's properties, which is where I learned things like the SGP30's siloxane poisoning or the op-amp's single-supply swing limits.

11. References

Standards and sources

ASTM E644 (thermistor characterization); ASHRAE Standard 62.1 (ventilation / indoor CO_2); ISO 7730 (PMV/PPD comfort); ASHRAE Standard 55 (thermal comfort, 20–26 °C); National Sleep Foundation (sleep temperature, 16–19 °C); EPA / AAFA (humidity guidance for asthma, 30–50 %RH); Sterling et al. 1985 (humidity 40–60 %RH); Strom-Tejsen et al. 2016 (CO_2 and sleep quality); WHO indoor air-quality guidelines (VOCs and asthma); German UBA TVOC tiers. Datasheets on file: Bosch BME280 (BST-BME280-DS002), Sensirion SGP30, Texas Instruments TL071, Texas Instruments LM2596, Espressif ESP32.

Human references

Dr. Trudgen via eLC lectures

AI references

Claude

- **One prompt:** "I want to make three different scores for air quality - like one for Asthma, one for Sleep quality, and one for Allergies or something. Can we do that and if yes, start looking for valid sources for how each measurand would contribute to each score. Include the sources you find."
- **(Right:** Dissuaded me from pursuing an 'Allergies Index' because allergies are caused by particulates which my system can't measure. Claude recommended a 'Freshness Index' instead that be weighted almost wholly on air-chemistry.)
- **Wrong:** Of eight sourced thresholds, two were numerically off (the sleep-temperature range and the ASHRAE 55 range) and had to be corrected against the primary sources.

Appendix

A. Final firmware. See attachment: airquality_CP5.ino (Structure: a one-time setup (I²C/LCD init, SGP30 baseline restore from flash, pad-baseline learning, WiFi) followed by five independently timed tasks in loop() - fast 200 ms (pad + thermistor + Kalman), gas 1 s (BME280 + SGP30 + fusion), baseline-hold 60 s, LCD, and ThingSpeak 20 s.)

B. Datasheets: Bosch BME280, Sensirion SGP30, TI TL071, TI LM2596, Espressif ESP32.

C. Final bill of materials.

Part	Qty	Cost	Link	Notes
ESP32 dev board (CP2102)	1	$9.39	[link]	µC, WiFi, touch peripheral, ADC
BME280 breakout	1	$8.99	[link]	Temp/%RH/Pressure, pre-packaged #1
SGP30 breakout	1	$12.49	[link]	eCO_2/TVOC, pre-packaged #2
10 kΩ NTC thermistor (10-pk)	1	$7.99	[link]	RAW sensor; β fitted 3992 K
TL071CP op-amp	1	$0.51	[link]	Thermistor front-end gain stage
Resistors R_1–R_8	8	$0.80	[link]	Front-end network
Capacitors C_1–C_3	3	$0.30	[link]	Filters and decoupling
LM2596 buck module (2-pk)	1	$4.99	[link]	Battery's ~8.4V to a regulated 5.0 V
6×AA NiMH + charger	1	$15.29	[link]	~20 Wh; more energy than 9V alternative
Copper tape	1	$7.99	[link]	Homemade capacitive sensor
20×4 LCD w/ I²C backpack (2-pk)	1	$8.99	[link]	Menu UI; 2pk is same price as single
Battery holder (6×AA)	1	$6.59	[link]	6 AA 9V case with leads
Tupperware enclosure (3-pk)	1	$3.49	[link]	Link is close but not exact: 3pk from Kroger

Total ≈ $87.81.

D. Raw data: See attachment: "RawData _ThingSpeak_FullExport.csv" (Field key: Field 1 = BME280 Temp, Field 2 = Humidity % RH, Field 3 = Pressure hPa, Field 4 = Thermistor (Kalman) Temperature, Field 5 = eCO_2 ppm, Field 6 = TVOC ppb, Field 7 = Asthma Index /100, Field 8 = Sleep Index/100)